Safe to spend this month

Epic: EPIC-018 · **Requirements:** A-23, A-19, A-07, A-22, A-21 · **Tasks:** #215 (use case #216, dashboard card #217)

> **Status:** the use case and its tests (#216) and the dashboard card with its explainer sheet (#217) are in. TODO(M4) add a screenshot of the card once it is captured on a device.

Problem

Before an impulsive purchase Kavindu asks "how much can I spend this month without hurting the MacBook goal?" (**A-23**, **A-19**). His income is irregular, so the answer cannot be a fixed budget bar (**A-07**).

Formula

All figures are LKR minor units for the current month in `Asia/Colombo`.

```
safeToSpend = receivedIncome - committed - goalSaving - discretionarySpent
committed   = committed expenses recorded this month
            + expected amount of every recurring template due this month and not yet confirmed
```

Term	Source	Rule
`receivedIncome`	`GetMonthlySummaryUseCase`	Actual income only. Expected or estimated income never counts (**A-22**). A crypto loss lowers it.
committed (recorded)	`GetSpendingBreakdownUseCase`	The `COMMITTED` share of actual expenses.
committed (reserved)	`RecurringRepository`	Unpaused templates whose `nextDueDate` falls in this month. Their expected amount is held back before the bill is paid.
`goalSaving`	`GetGoalProgressUseCase` + `ObserveActiveGoalUseCase`	What the goal still needs per remaining month, counting only what was saved **before this month**: (target - `savedBeforeThisMonth`) / `monthsLeft`, rounded up. 0 with no goal, or once last month's saving already reached the target.
`discretionarySpent`	`GetSpendingBreakdownUseCase`	The `DISCRETIONARY` share of actual expenses.

Transfers are not income or expense in any input, so they never move the figure (**A-21**).

`perDay = safeToSpend / daysLeft`, only when the state is `AVAILABLE`. `daysLeft` counts today, so it is 1 on the last day. Division uses `BigDecimal` with `HALF_UP`.

States

Checked in this order:

State	When	Card copy
`NOT_ENOUGH_DATA`	No income recorded this month and no expected salary set	First-run hint

State	When	Card copy
AVAILABLE	safeToSpend > 0	"LKR 18,400 safe to spend - approx LKR 2,300/day for 8 days"
WAITING_FOR_SALARY	safeToSpend <= 0, an expected salary is set, and no salary is recorded this month	"Waiting for salary - we'll update this when it's in"
FULLY_USED	Anything else with safeToSpend <= 0	"This month's plan is fully used - LKR 3,200 over"

FULLY_USED is neutral in colour, never alarm red (**A-23**). WAITING_FOR_SALARY exists so a bill reserved early in the month does not show a scary negative number.

Decisions that go beyond the ticket text

- **Recurring templates come from RecurringRepository, not GetDueRecurringUseCase.** That use case lists only items already due, but the reserve must include bills due later in the month.
- **Only templates whose nextDueDate is in the current month are reserved.** A cursor still in an earlier month is a catch-up booked in that month when confirmed (DashboardViewModel.confirm), so it does not touch this month. One consequence: a template that is two or more months behind reserves nothing until it is caught up.
- **Goal saving does not use GoalProgress.requiredMonthlyMinor as is.** That figure is built from the saved total including this month's received - spent, so it moves with this month's entries. Safe to spend already subtracts this month's spending itself, so each expense would lower the result twice (and each income raise it twice; in the deadline month the effect is 2x). Backing this month's net out of the saved total removes that. A side effect: the "goal" number in the explainer can differ slightly from the "required per month" on the goal card, because the card still counts this month. A review on PR #259 caught this.
- **WAITING_FOR_SALARY needs an expected salary.** A user with no salary set, who has other income and has spent it all, sees FULLY_USED, because there is no salary to wait for.
- **The model carries daysLeft** so the card can say "for 8 days" without recomputing the date.

Edge cases

Case	Behaviour
Confirmed recurring entry (fixed amount)	The expense is recorded as committed and the cursor moves to next month, so the bill leaves the reserve. The figure is unchanged, with no double count.
Confirmed recurring entry (variable amount)	Same swap, but the recorded amount replaces the reserved one, so the figure moves by the difference. A bill reserved at LKR 34,000 and confirmed at LKR 37,500 lowers safe to spend by LKR 3,500; confirmed at LKR 30,000 it raises it by LKR 4,000. Still no double count.
Paused template	Never reserved.
Template due next month	Not reserved.
No goal	goalSaving is 0.
Exactly zero left	FULLY_USED, over by 0.
Last day of the month	daysLeft = 1, perDay equals the whole amount.

Case	Behaviour
Midnight in Colombo	The month and `daysLeft` roll over at Colombo midnight, not UTC.

Dashboard card (#217)

- It sits directly under the "This month" card, and also in the empty first-run month, where it shows the first-run hint.
- `DashboardViewModel` exposes it as `DashboardUiState.safeToSpend`. The screen only renders it.
- The figure is for today's month, so it is hidden while the user pages to a past month. A failure hides only the card, never the rest of the dashboard.
- Tapping the card opens a small sheet: "Received X - committed X - goal X - spent X", then the result, then one plain line on each term. The numbers come from `SafeToSpend.parts`.
- Every state uses the normal card colours. Nothing is red, even when the month is used up (**A-22**).

Live updates

The use case combines Room-backed flows, so it re-emits after any entry, recurring confirm or goal change, and at each midnight.

Tests

`GetSafeToSpendUseCaseTest` covers every state, no goal, a reserved bill, a confirmed bill not counted twice, transfers, a crypto loss, the last day of the month, rounding and live updates.